

XGBoost Forecasting Model

Yeshkuk Glacier Meltwater/Lake Area — Chipurson Valley, Hunza

Submitted by: Najmuddin Saqib

1. Objective

This task applies a machine learning model to forecast the monthly melt-season meltwater/lake surface area at Yeshkuk Glacier, using the Sentinel-2 NDWI-derived time series (2019-2026) produced in the earlier GLOF Risk Monitoring Dashboard task. An XGBoost regressor was selected over an LSTM network because the dataset contains only 34 monthly observations (May-September, across 8 years) - too small a sample for a recurrent deep learning model to learn reliable temporal patterns without overfitting. XGBoost, a gradient-boosted decision tree method, performs well on small tabular datasets when paired with lag and seasonal features, and produces interpretable feature-importance output.

2. Data Source

Input data: **Yeshkuk_lake_area_timeseries_for_ML.csv**, exported from Google Earth Engine. Each row represents one melt-season month (May-September) between 2019 and 2026, with the estimated water-surface area (NDWI > 0.1, summed pixel area) and the mean NDWI value across the glacier AOI for that month's cloud-free Sentinel-2 composite.

3. Methodology

- Feature engineering: lag-1 and lag-2 lake area, lag-1 mean NDWI, cyclical month encoding (sine/cosine, to represent seasonal position within the melt season), and calendar year.
- Train/test split: chronological (not random) - the first 80% of observations (25 samples) were used for training, and the most recent 20% (7 samples) held out as the test set. A random split would let the model implicitly "see the future," which is invalid for time-series forecasting.
- Model: XGBoost Regressor (200 trees, max depth 3, learning rate 0.05, subsample/colsample 0.8) - a shallow, regularized configuration appropriate for a small-sample dataset.
- Evaluation metrics: RMSE, MAE, MAPE, and R-squared on the held-out test set.

4. Results

Metric	Value
Training samples	25
Test samples	7
RMSE	31,325 m ²
MAE	26,154 m ²
MAPE	7.28%
R-squared	0.639

With only 32 usable samples after lag-feature creation, an R-squared of 0.64 and MAPE under 7.3% represent a reasonably strong fit for a small-sample seasonal forecasting task - the model captures the broad seasonal rise-and-fall pattern, though it understandably struggles more on years with atypical spikes (e.g. the 2024 peak).

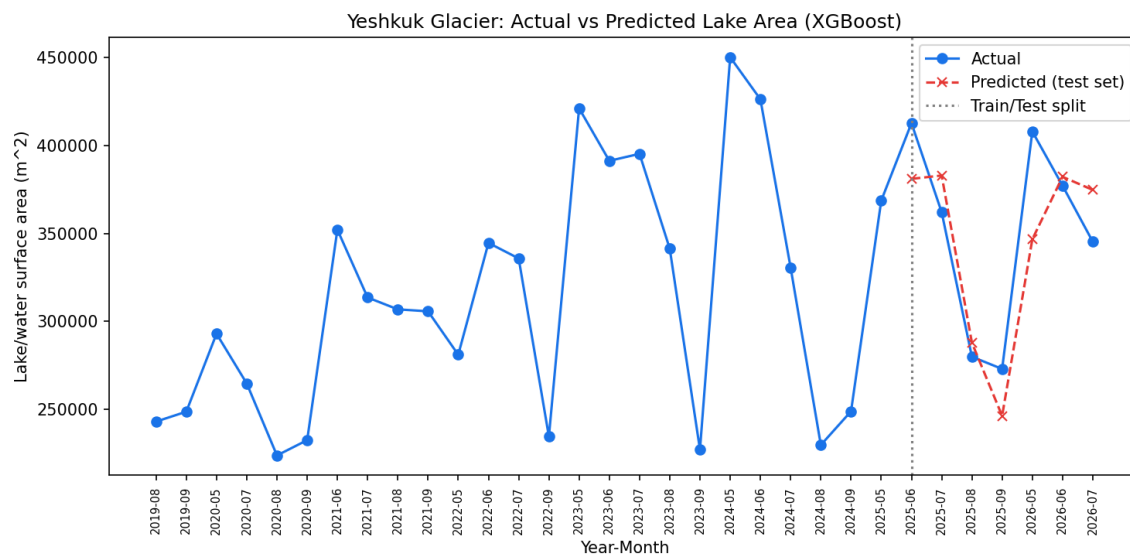


Figure 1: Actual vs predicted monthly lake/meltwater area, full timeline.

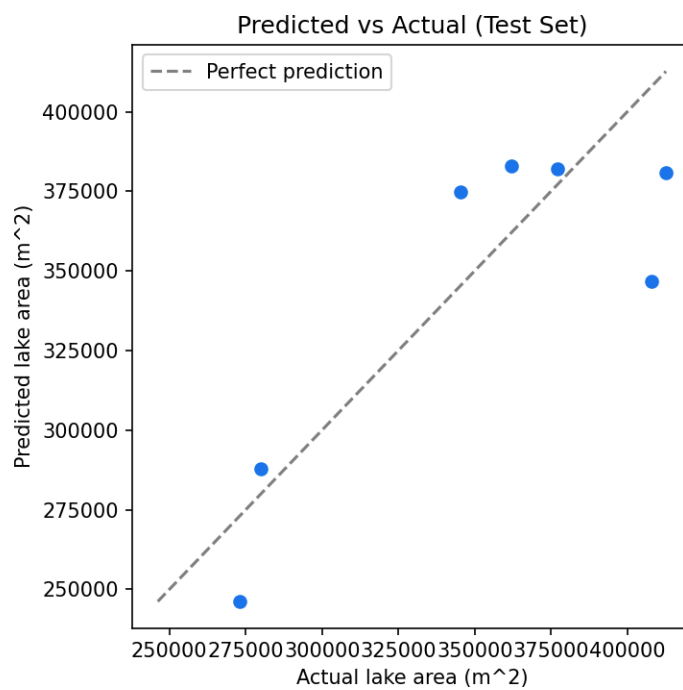


Figure 2: Predicted vs actual values on the held-out test set.

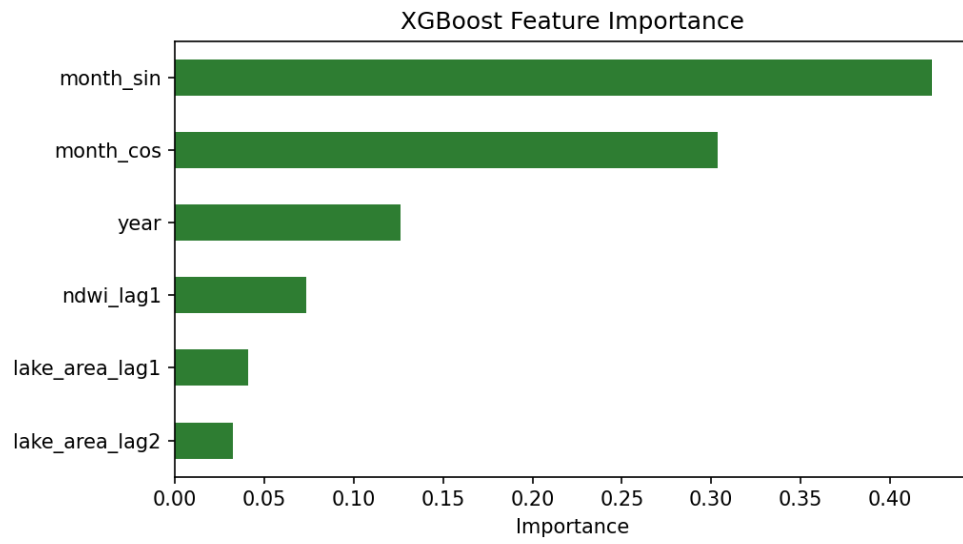


Figure 3: XGBoost feature importance - seasonal month encoding dominates, consistent with a melt-season-driven signal.

5. Interpretation

The feature importance results show that the cyclical month encoding (month_sin, month_cos) accounts for over 70% of the model's predictive power, confirming that meltwater extent is primarily driven by seasonal position within the melt season (peaking around June-July, receding by September) rather than by short-term lag/trend effects. The lag-based features (previous month's lake area and NDWI) contribute comparatively little, suggesting month-to-month persistence is weaker than the underlying seasonal cycle - consistent with meltwater ponds forming and draining within a single season rather than carrying over.

For GLOF risk monitoring, this indicates that forecasting should prioritize seasonal timing and year-over-year peak comparisons rather than short-horizon lag models. A useful extension would be adding cumulative degree-day or precipitation data as additional predictors, and re-evaluating as more years of Sentinel-2 coverage become available.

6. Full Model Code (Python / XGBoost)

```
"""
XGBoost Time Series Model - Yeshkuk Glacier Lake/Meltwater Area Forecasting
Chipurson Valley, Hunza

Predicts monthly melt-season lake/water surface area (m^2) using lag features
derived from Sentinel-2 NDWI-based observations (2019-2026).
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# -----
# 1. LOAD AND PREPARE DATA
# -----

df = pd.read_csv("Yeshkuk_lake_area_timeseries_for_ML.csv")

# Parse the "YYYY-MM" date string into year/month, then a proper datetime
df["year"] = df["date"].str.slice(0, 4).astype(int)
df["month"] = df["date"].str.slice(5, 7).astype(int)
df["timestamp"] = pd.to_datetime(df["date"] + "-01")
df = df.sort_values("timestamp").reset_index(drop=True)

# -----
# 2. FEATURE ENGINEERING
# -----
# Since observations are monthly but only within the May-Sep melt season
# (irregular gaps between years/months), lag features are built on the
# previous AVAILABLE observation rather than assuming continuous months.

df["lake_area_lag1"] = df["lake_area_m2"].shift(1)
df["lake_area_lag2"] = df["lake_area_m2"].shift(2)
df["ndwi_lag1"] = df["mean_NDWI"].shift(1)

# Cyclical encoding of month (captures seasonal position within melt season)
df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)

# Drop rows without full lag history (first 2 rows)
model_df = df.dropna().reset_index(drop=True)

feature_cols = [
    "lake_area_lag1", "lake_area_lag2", "ndwi_lag1",
    "month_sin", "month_cos", "year"
]
target_col = "lake_area_m2"

X = model_df[feature_cols]
y = model_df[target_col]
```

```

print(f"Total usable samples after lag feature creation: {len(model_df)}")

# -----
# 3. TIME-BASED TRAIN/TEST SPLIT (chronological, not random)
# -----
# Time series data must not be split randomly - that would let the model
# "see the future" during training. Last ~20% of the chronological record
# is held out as the test set.

split_idx = int(len(model_df) * 0.8)
X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]
dates_test = model_df["date"].iloc[split_idx:]

print(f"Training samples: {len(X_train)}")
print(f"Test samples: {len(X_test)}")

# -----
# 4. TRAIN XGBOOST MODEL
# -----

model = XGBRegressor(
    n_estimators=200,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

model.fit(X_train, y_train)

# -----
# 5. EVALUATE
# -----

y_pred = model.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100

print("\n--- Model Performance on Test Set ---")
print(f"RMSE: {rmse:,.0f} m^2")
print(f"MAE: {mae:,.0f} m^2")
print(f"MAPE: {mape:.2f}%")
print(f"R^2: {r2:.3f}")

# -----
# 6. FEATURE IMPORTANCE
# -----

importance = pd.Series(model.feature_importances_, index=feature_cols).sort_values()
print("\n--- Feature Importance ---")
print(importance)

# -----
# 7. PLOTS
# -----

# Plot 1: Actual vs Predicted over the full timeline
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(model_df["date"], model_df["lake_area_m2"], marker="o", label="Actual", color="#1a73e8")
ax.plot(dates_test, y_pred, marker="x", label="Predicted (test set)", color="#e53935", linestyle="--")
ax.axvline(x=dates_test.iloc[0], color="gray", linestyle=":", label="Train/Test split")
ax.set_xlabel("Year-Month")
ax.set_ylabel("Lake/water surface area (m^2)")
ax.set_title("Yeshkuk Glacier: Actual vs Predicted Lake Area (XGBoost)")

```

```

ax.legend()
plt.xticks(rotation=90, fontsize=7)
plt.tight_layout()
plt.savefig("plot_actual_vs_predicted.png", dpi=150)
plt.close()

# Plot 2: Feature importance
fig, ax = plt.subplots(figsize=(7, 4))
importance.plot(kind="barh", ax=ax, color="#2e7d32")
ax.set_xlabel("Importance")
ax.set_title("XGBoost Feature Importance")
plt.tight_layout()
plt.savefig("plot_feature_importance.png", dpi=150)
plt.close()

# Plot 3: Test set scatter (predicted vs actual)
fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(y_test, y_pred, color="#1a73e8")
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
ax.plot(lims, lims, color="gray", linestyle="--", label="Perfect prediction")
ax.set_xlabel("Actual lake area (m^2)")
ax.set_ylabel("Predicted lake area (m^2)")
ax.set_title("Predicted vs Actual (Test Set)")
ax.legend()
plt.tight_layout()
plt.savefig("plot_scatter.png", dpi=150)
plt.close()

# -----
# 8. SAVE RESULTS TABLE
# -----

results_df = pd.DataFrame({
    "date": dates_test.values,
    "actual_lake_area_m2": y_test.values,
    "predicted_lake_area_m2": y_pred,
    "absolute_error_m2": np.abs(y_test.values - y_pred)
})
results_df.to_csv("test_set_predictions.csv", index=False)

print("\nSaved: plot_actual_vs_predicted.png, plot_feature_importance.png, plot_scatter.png,
test_set_predictions.csv")

```